

Enunciado TPE1

Trabajo Práctico Especial - Primera entrega

Objetivo

Crear una aplicación web con Java 21 utilizando metodología **Scrum** y dejarla productiva en un servidor web.

Tecnología a utilizar

- La aplicación debe poseer una arquitectura según **Domain Driven Design**, aplicando el patrón de inyección de dependencias a través de **Spring**.
- Se deben escribir test unitarios que den cobertura suficiente al sistema.
- La capa web debe estar implementada utilizando los patrones MVC y Front Controller, a través de **Spring Web MVC**. La vista debe estar compuesta por archivos **JSP** con **JSTL** (no deben contener código Java).
- Para la persistencia de los objetos de dominio se debe utilizar **JDBC**.
- Para la administración y construcción del proyecto se debe utilizar **Maven**.
- La aplicación debe almacenar los datos en una base de datos **PostgreSQL**.
- El versionado de código debe realizarse mediante el repositorio **GIT** provisto por la cátedra.

Enunciado

Los alumnos construirán aplicaciones según el proyecto propuesto previa aprobación por parte de la cátedra. Todo proyecto igualmente debe contemplar:

- Diferentes niveles de acceso por permisos de usuario
- Internacionalización del contenido
- Manejo de logging
- Mailing

- Queries no triviales (búsquedas verticales o similares)
- Manejo de imágenes

Iteraciones

Los alumnos deberán trabajar con iteraciones de 2 semanas (salvo excepciones). No todo cierre de iteración será evaluado, pero **la demo de fin de iteración es obligatoria**. Estas demos servirán como punto de control para el propio equipo y la cátedra; siendo una oportunidad para que el equipo haga una retrospectiva y ajuste su forma de trabajo para conseguir mejores resultados.

Cada equipo deberá proponer qué compromete iteración a iteración en función del tiempo que estén dispuestos a asignar a la materia y la priorización que hayan realizado del backlog. Iteración a iteración se espera igualmente un nivel de avance **mínimo** que consta de:

- Incorporación **completa** del feedback de la cátedra en la entrega anterior
- La iteración de los flujos entregados en iteraciones pasadas conforme se completan y complejizan
- La iteración de nuevos flujos core del proyecto (es decir, no sólo ABMs)

Habrás así un cierre de iteración el **08/04**, otro el **22/04** y finalmente, la entrega final a ser evaluada el **04/05**.

Consideraciones

Independientemente del enunciado elegido, se espera una implementación iterativa que permita la puesta en producción del sistema desde la etapa más temprana posible.

Será responsabilidad del equipo, iteración a iteración, realizar todas las migraciones de base de datos necesarias para garantizar que **no se pierda información alguna**.

La planificación y seguimiento del trabajo se realizará utilizando el proyecto de Plane provisto por la cátedra.

Entrega

La totalidad del código debe estar disponible en el repositorio git provisto por la cátedra. Se espera que el mismo **no incluya** archivos innecesarios a estarse

usando **Maven**, como ser archivos de configuración de Eclipse, dependencias o archivos generados por **Maven** mismo.

Se espera que el proyecto pueda producir un **war** usable con sólo utilizar `mvn clean package` y que al mismo ser deployado en un **Application Container** existiendo una base de datos **PostgreSQL** con permisos adecuados, se encargue de generar todas las tablas necesarias para ser utilizado.

Adicionalmente, la aplicación debe estar funcionando correctamente en el servidor provisto por la cátedra.

En caso de que la aplicación requiera algún tipo de autenticación para accederla, debe existir un archivo `README.md` que indique claramente credenciales para al menos un usuario con cada nivel de acceso existente en la aplicación.

La entrega se realizará **mediante un email a la cátedra con el hash al commit deployado** a más tardar el **04/05 a las 19:00** debiéndose hacer una demo ese mismo día en el horario de clase.

Entregas intermedias:

En caso de no cumplir con lo requerido para cada sprint, se pedirá una entrega intermedia para la clase inmediata siguiente.

Cada vez que un grupo llega a entrega intermedia se puede penalizar con hasta 1 punto.

Evaluación

En cada iteración se evaluará el grado de cumplimiento de los requerimientos y la calidad del desarrollo (funcionamiento, usabilidad, diseño de objetos).

La cátedra hará una corrección no sólo funcional, sino que también auditará código, analizará commits y contribuciones individuales de cada miembro del equipo. Así, la cátedra se reserva el derecho de evaluar en forma individual a un alumno si lo considera necesario.

Se evaluará

En la entrega final

- Apropiado nivel de Unit Testing y calidad de los tests unitarios. Su incumplimiento se penalizará con hasta 1 punto.

- Forma de entrega acorde a lo enunciado. Su incumplimiento se penalizará con hasta 1 punto.
- Funcionalidad y usabilidad del producto entregado. Se podrá penalizar hasta 2 puntos por fallos en este frente.
- Correcta arquitectura e implementación. Se podrá penalizar hasta 4 puntos por problemas derivados de no seguir Domain Driven Design, no hacer una correcta separación de capas y responsabilidades o no seguir Effective Java u otras buenas prácticas de programación.

Se evaluará en cada sprint

- Cumplimiento de la metodología SCRUM. Uso de Plane, priorización de la funcionalidad, valor aportado al usuario en cada iteración, etc. Se evaluará en cada sprint, restando 0.5 puntos por cada sprint intermedio.
- Funcionalidad y usabilidad del producto entregado. Se podrá penalizar hasta 1 punto por sprint por fallos en este frente.
- Perdidas de información. Se penalizarán con 2 puntos.

Errores no recuperables

Se considerará reprobado sin posibilidad de recuperar el trabajo práctico en caso de:

- No realizar cualquier demo de fin de iteración o no tener cambios a presentar.
- No cumplir con los requerimientos técnicos establecidos en el enunciado.
- Inconsistencias en el hash del commit presentado.

Bibliografía

Se recuerda a los alumnos la **fuerte recomendación** de leer:

- Effective Java, 3rd Edition de Joshua Block
- Domain Driven Design, de Evans